preliminary evaluation of ChatGPT's potential and limitations in recommender systems.

3 PROBING CHATGPT FOR RECOMMENDATION CAPABILITIES

In this section, we leverage prompts to adapt point-wise, pair-wise, and list-wise ranking tasks, enabling off-the-shelf LLMs to effectively tackle these tasks.

3.1 Three Ranking Capabilities in Recommender Systems

The core objective of personalized recommendation is to rank candidate items based on user preferences. To accomplish this, current learning-to-rank (LTR) methods empower different capabilities to recommender systems via corresponding loss functions, including point-wise ranking capability, pair-wise ranking capability and list-wise ranking capability [18]. Formally, given a user $u \in \mathcal{U}$ and k candidate items $\{i_1, i_2, \cdots, i_k\} \subset \mathcal{I}$, each user-item pair's representation is encoded as $\mathbf{x}_{u,i}$. The above three capabilities can be formulated as follows:

Definition 1. *(point-wise ranking capability) The recommender system learns to predict the preference score of each item i for each user u via a point-wise scoring function* $\Phi_{\text{point}}(\cdot)$: $s(i \mid u) = \Phi_{\text{point}}(\mathbf{x}_{u,i})$ *The preference score s is then used to rank the items for each user. The common used loss function in point-wise ranking includes mean squared error (MSE) [31] and binary cross entropy (BCE) [12].*

Definition 2. *(pair-wise ranking capability) The recommender system learns to compare pairs of items* i_m *and* i_n *for each user u and predict which item is preferred via a pair-wise scoring function* $\Phi_{\text{pair}}(\cdot)$: $s(i_m \succ i_n \mid u) = \Phi_{\text{pair}}(\mathbf{x}_{u,i_m}, \mathbf{x}_{u,i_n})$ *The system then ranks the items based on the relative preference score s. The pairwise hinge loss [14] or Bayesian personalized ranking loss (BPR) [30] are the typical loss functions utilized in pair-wise ranking.*

Definition 3. *(List-wise ranking capability) The recommender system learns to directly predict the preference score of a list of items* $\{i_1, i_2, \cdots, i_k\}$ *for each user u via a list-wise scoring function* $\Phi_{\text{list}}(\cdot)$: $s(i_1 \mid u), s(i_2 \mid u), \cdots, s(i_k \mid u) = \Phi_{\text{list}}(\mathbf{x}_{u,i_1}, \mathbf{x}_{u,i_2}, \cdots, \mathbf{x}_{u,i_k})$ *The system then sorts the items based on the predicted scores. The list-wise loss, e.g., sampled softmax loss [7] is employed to optimize the recommendation model.*

3.2 Reformulate and Adapt Recommendation with Prompts

To obtain above capabilities of recommendation, current recommendation models employ corresponding loss functions for supervised learning. However, the supervised learning schema often fails in data sparsity scenarios (e.g., cold start problems [10] and long-tailed items [24]). In contrast, LLMs have a stronger generalization capability in these data sparsity scenarios and achieve promising performances in few-shot and even zero-shot tasks. In this empirical study, we assume that LLMs already have the above three capabilities, and all we need to do is to trigger these capabilities through prompt tuning. To this end, we adopt the recent successful practice of in-context learning [3] and instruction tuning [6], and we express the aforementioned three capabilities as three tasks with domain-specific prompts.

Figure 1 illustrates how we employ prompt tuning to elicit three ranking capabilities from LLMs. As shown in Figure 1 (left), our prompt consists of three components: (i) *Task description I* refers to the process of enabling the LLM to comprehend the particular domain in which it is required to perform recommendation tasks. The task description is designed to be domain-aware, which enhances LLM's perception of pertinent knowledge. (ii) *Demonstration examples* $\mathcal{D} = \{f(\mathbf{h}_1, \mathbf{c}_1, \mathbf{y}_1), \cdots, f(\mathbf{h}_N, \mathbf{c}_N, \mathbf{y}_N)\}$ (i.e., N-shot in-context learning), where $\mathbf{h}$ denotes the historical interacted items of a user, $\mathbf{c}$ denotes the candidate items which need to be ranked, $\mathbf{y}$ denotes the predictions given by LLMs, and $f(\cdot)$ is the function for transforming the examples into designed prompt templates. The demonstration examples facilitate the LLM's comprehension of the current task. (iii) *New input query* of a given user $f(\mathbf{h}', \mathbf{c}' \mid u)$, which needs to be answered by LLMs. For three ranking tasks, the corresponding candidate items $\mathbf{c}$ are constructed as follows:

$$\mathbf{c} = \begin{cases} \{i\} & \text{for point-wise ranking capability,} \\ \{i_m, i_n\} & \text{for pair-wise ranking capability,} \\ \{i_1, i_2, \cdots, i_k\} & \text{for list-wise ranking capability.} \end{cases}$$

As depicted in Figure 1 (right), LLMs will utilize the different ranking capabilities elicited through different prompts to make predictions $\hat{y}'$:

$$\begin{aligned} \hat{y}'_i &= LLM_{\text{point}}\left(I, \mathcal{D}, f(\mathbf{h}', \mathbf{c}' \mid u)\right), \\ \hat{y}'_{i_m \succ i_n} &= LLM_{\text{pair}}\left(I, \mathcal{D}, f(\mathbf{h}', \mathbf{c}' \mid u)\right), \\ \hat{y}'_{i_1}, \hat{y}'_{i_2}, \cdots, \hat{y}'_{i_k} &= LLM_{\text{list}}\left(I, \mathcal{D}, f(\mathbf{h}', \mathbf{c}' \mid u)\right). \end{aligned}$$

Then the output answer will be checked manually, and the valid answers will be utilized for further evaluation, while the invalid answers will be excluded. For more details about the prompts, please refer to the link[1].

4 EXPERIMENTS

In this section, we conduct experiments to evaluate ChatGPT and GPT-3.5s to answer the following research questions:

RQ1: How do these LLMs perform on different ranking capabilities across various recommendation domains?

RQ2: How do the LLMs-based recommenders compare with traditional collaborative filtering methods?

RQ3: How much cost of these LLMs-based recommenders on different ranking capabilities?

RQ4: How does the number of prompt shots affect the performance of LLMs-based recommenders?

4.1 Experimental Settings

4.1.1 Datasets. To better probe the different capabilities of ChatGPT and GPT-3.5s on personalized recommendation, we conducted evaluations on datasets from four different domains.

Movie: We use the widely-adopted MovieLens-1M[2] dataset that contains 1M user ratings for movies.

[1] https://github.com/rainym00d/LLM4RS/blob/main/assets/prompts.pdf
[2] https://grouplens.org/datasets/movielens/1m/